

Permisos en el frontend de Conciliación Bancaria

Sep 28, 2026 · @Irving Canul

Resumen

Este documento explica el sistema de permisos del frontend de **Conciliación Bancaria**: qué roles y permisos existen, de dónde salen (backend real vs. mock local), cómo se resuelven en cada sesión, y cómo se usan para mostrar/ocultar pantallas y acciones. Está pensado para quien vaya a tocar el módulo de autenticación, administración de usuarios, o agregar una pantalla nueva que deba restringirse por rol.

Modelo de roles

Los roles (`RoleId` , en `features/user-management/data/user-management.model.ts`) son **exactamente los que siembra el backend real** (`app_role.code`) — no un vocabulario inventado por el frontend. Cada uno da acceso a un módulo completo de la app (salvo uno, ver tabla):

Rol (<code>RoleId</code>)	Etiqueta en la UI	Módulo al que da acceso
ADMIN	Administrador	Todos (Resumen de venta, Conciliación bancaria, Administración de usuarios)
ALTAS	Encargado de altas	Solo Administración de usuarios
CONTABILIDAD	Contabilidad	Solo Resumen de venta
TESORERIA	Tesorería	Solo Conciliación bancaria
COSTOS	Costos	Ninguno todavía — rol real del backend sin pantalla asignada en esta app

Cada usuario puede tener **varios roles a la vez** (`AppUser.roleIds: RoleId[]`) — sus permisos efectivos son la unión de los `defaultPermissions` de todos sus roles (ver siguiente sección).

El catálogo completo vive en `ROLES: RoleDef[]` (mismo archivo) — `label`, `description` y `defaultPermissions` por rol. `ROLE_LABEL` / `ROLE_OPTIONS` se derivan de ahí para mostrarlos en la UI (chips, selects).

Modelo de permisos

Los permisos (`PermissionKey`) son más finos que los roles — acciones concretas dentro de un módulo. Son 8, agrupados por pantalla (`PERMISSIONS: PermissionDef[]` , mismo archivo):

Permiso (<code>PermissionKey</code>)	Etiqueta	Módulo (<code>group</code>)
<code>view_dashboard</code>	Ver resumen de venta	Resumen de venta
<code>view_reconciliation</code>	Ver conciliación	Conciliación bancaria
<code>manage_differences</code>	Gestionar diferencias	Conciliación bancaria
<code>import_settlements</code>	Importar liquidaciones (CSV)	Conciliación bancaria
<code>export_reports</code>	Exportar reportes	Conciliación bancaria
<code>manage_users</code>	Administrar usuarios	Administración de usuarios
<code>manage_roles</code>	Administrar roles y permisos	Administración de usuarios
<code>view_audit_log</code>	Ver historial y auditoría	Administración de usuarios

Permisos por defecto de cada rol (`defaultPermissionsForRoles(roleIds)` calcula la unión cuando alguien tiene varios roles):

Rol	Permisos por defecto
<code>ADMIN</code>	los 8
<code>ALTAS</code>	<code>manage_users</code> , <code>manage_roles</code> , <code>view_audit_log</code>
<code>CONTABILIDAD</code>	<code>view_dashboard</code>
<code>TESORERIA</code>	<code>view_reconciliation</code> , <code>manage_differences</code> , <code>import_settlements</code> , <code>export_reports</code>
<code>COSTOS</code>	ninguno (<code>[]</code>)

Los permisos son el nivel que de verdad consulta la UI (`hasPermission(key)`) — el rol es solo el punto de partida al asignar o cambiar de rol a un usuario; un admin puede después ajustar sus permisos por encima/debajo de ese default (ver "Administración de permisos" más abajo).

Cómo se resuelve la sesión

`AccessControlService` (`features/auth/data/access-control.service.ts`) es el único lugar que resuelve "qué puede ver/hacer" el usuario logeado — nada más en la app debería leer permisos por su cuenta. Vive **separado** de `AuthService` a propósito: `AuthService` solo sabe quién inició sesión (email, roles del backend, tokens), nunca permisos de negocio.

`permissions` (el computed que todo lo demás consulta vía `hasPermission(key)`) tiene **dos fuentes**, en este orden de prioridad:

1. **`AppUser.permissions` del mock local**, cuando la sesión está vinculada a un registro de `user-management` (`currentAppUser()`), resuelto por email — ver `UserManagementService.findUserByEmail`). Esta es la fuente para las 4 cuentas demo y para cualquier usuario del mock: respeta ajustes manuales que un admin haya hecho sobre ESE usuario en "Seguridad y acceso" (no siempre coincide con el default de su rol).
2. **`defaultPermissionsForRoles(auth.currentUser()?.roles)`**, cuando NO hay ese vínculo — se calculan los permisos por defecto a partir de los roles REALES que devolvió el backend en el login. Esto es lo que permite que **cualquier cuenta del backend real** (no solo las 4 de demo) aterrice en la pantalla correcta según su rol, aunque no exista un registro suyo en el mock de administración de usuarios (todavía no hay backend real para ESE módulo).

`mustChangePassword` sigue el mismo criterio de dos fuentes (mock O el flag que trajo el login real, lo que sea `true` primero).

`homeRoute()` decide a dónde aterriza alguien tras iniciar sesión (y a dónde rebota si entra a una pantalla que no le corresponde): revisa permisos en un orden fijo —

`view_dashboard` → `/dashboard`, si no `view_reconciliation` → `/conciliacion`, si no `manage_users` → `/usuarios`, y si no tiene ninguno de los tres, `/login` (le pasa a `COSTOS` y a cualquier cuenta sin permisos reconocidos).

Guards de ruta

Tres guards, cada uno resuelve una sola cosa (`features/auth/data/*.guard.ts`):

Guard	Dónde se aplica	Qué exige	Si falla
<code>authGuard</code>	Shell completo (todo lo que cuelga de <code>'</code> , ver <code>app.routes.ts</code>)	Sesión activa, y sin cambio de contraseña pendiente	Sin sesión → <code>/login</code> ; con cambio pendiente → <code>/cambiar-password</code>
<code>mustChangePasswordGuard</code>	Solo <code>/cambiar-password</code>	Sesión activa Y cambio de contraseña realmente pendiente	Sin sesión → <code>/login</code> ; sin nada pendiente → <code>homeRoute()</code> (evita que alguien la visite por curiosidad o se quede ahí tras completarla)
<code>permissionGuard</code>	Cada ruta hija de Shell que lo necesita, vía <code>data</code> : <code>{ permission: '...' }</code> (<code>/dashboard</code> , <code>/conciliacion</code> , <code>/usuarios</code> y sus sub-rutas)	El permiso declarado en esa ruta	Redirige a <code>homeRoute()</code> (nunca deja una pantalla en blanco)

Son complementarios, no se pisan: `authGuard` decide si puedes entrar a Shell EN GENERAL (autenticado + contraseña al día); `permissionGuard` , ya adentro, decide si puedes ver ESA pantalla específica. `/perfil` no declara `data.permission` — cualquier cuenta autenticada puede verla, sin importar su rol.

Dónde se consume en la UI

Además de los guards de ruta, `AccessControlService.hasPermission()` se consulta directo en tres lugares más, para que la navegación nunca ofrezca algo que el guard igual va a rechazar:

- **Menú lateral** (`core/layout/menu/menu.html`) — cada uno de los 3 links (Dashboard/Conciliación/Usuarios) está envuelto en su propio `@if (access.hasPermission('...'))`; un usuario sin ese permiso ni siquiera ve el link.
- **Buscador del Header** (`core/layout/header/header-search.util.ts`) — `SEARCHABLE_PAGES` declara un `permission?` opcional por página; el Header filtra ANTES de buscar por texto (`SEARCHABLE_PAGES.filter(page => !page.permission || access.hasPermission(page.permission))`), así que buscar "usuarios" no muestra el resultado si no puedes entrar ahí.
- **homeRoute()** (ver sección anterior) — tanto `Login` como `permissionGuard` lo usan para saber a dónde mandar a alguien que no puede ver lo que pedía.

No hay botones/acciones DENTRO de una pantalla que hoy estén condicionados por un `PermissionKey` distinto al de la ruta que los contiene — la granularidad actual es "pantalla completa", no acciones sueltas dentro de una misma pantalla (ver Limitaciones).

Administración de permisos

En `/usuarios/:userId`, pestaña "**Seguridad y acceso**" (`user-detail.ts / .html`), un admin puede cambiar los roles y permisos de otro usuario:

1. **Cambiar los roles asignados RESETEA los permisos a la unión de los defaults de esos roles** (`onRolesSelectChange` → `defaultPermissionsForRoles(roleIds)`) — es un punto de partida, no una regla que se recalcula sola después.
2. **Los permisos, una vez ahí, son editables uno por uno** (grid de checkboxes agrupado por módulo, `PERMISSION_GROUPS`) — el admin puede darle a alguien un permiso fuera del default de su rol, o quitarle uno que sí viene por defecto.
3. **"Guardar permisos" es un botón aparte** de "Guardar cambios" (información personal + organización) — `onSaveRolesAndPermissions()` llama `UserManagementService.changeRoles(id, roleIds, permissions)`, que persiste ambos y registra una entrada de auditoría (`role_changed` o `permissions_changed` , según si el SET de roles cambió o solo los permisos).

4. Esto solo aplica a usuarios del **mock local** de `user-management` (sin backend real de administración de usuarios todavía) — no cambia nada en el backend real de autenticación; para las 4 cuentas demo vinculadas por email, si se editan sus permisos aquí, esos SON los que usa `AccessControlService` (fuente #1, ver "Cómo se resuelve la sesión").

Limitaciones conocidas y pendientes

- **UserSummaryDto.permissions del backend real no se usa** — el catálogo de permisos (`app_permission` / `app_role_permission`) no tiene filas sembradas en ningún backend probado hasta ahora, así que esa lista siempre llega vacía. Solo `roles` se usa (vía `defaultPermissionsForRoles`).
- **COSTOS (rol real del backend) no tiene módulo propio en esta app** — una cuenta solo-COSTOS se autentica bien, pero cae a `/login` por falta de permisos. No se inventó a qué pantalla debería dar acceso; queda pendiente de una decisión de producto.
- **Granularidad = pantalla completa, no acciones sueltas** — de los 8 permisos, solo 3 (`view_dashboard` , `view_reconciliation` , `manage_users`) se consultan en algún `hasPermission()` real (rutas, menú, buscador). Los otros 5 (`manage_differences` , `import_settlements` , `export_reports` , `manage_roles` , `view_audit_log`) existen como datos asignables (checkboxes en "Seguridad y acceso") pero ninguna pantalla los consulta todavía para mostrar/ocultar una acción específica dentro de un módulo.
- **La vinculación sesión ↔ mock es por email, manual** — solo las 4 cuentas demo (`u00001` / `u00006` / `u00011` / `u00018`) tienen un email que coincide con lo sembrado en el backend real; cualquier otra cuenta real que se cree ahí no tendrá `AppUser` vinculado en `user-management` hasta que exista un backend real de administración de usuarios que los comparta.